PROJECT ASSIGNMENT DOCUMENT

RealTimeRx — DevSecOps Pipeline Modernization

Client Group → Contractor Group

Version 2.0 | All components run locally | No cloud services

Document type	Project Assignment Document (PAD)
Client group	Group __bs____
Contractor group	Group __none____
Handover date	_____
Stack	Python Flask · PostgreSQL · Docker · Jenkins · Git
Environment	Fully local — no cloud, no external services

1. The Incident — Why This Project Exists

Before reading the technical requirements, read this real incident based story. Every requirement in this document exists because of what happened below.

What actually broke — the real sequence of events

Day 1: MedSupply Co. changed batch numbers from MED-001 format to MED-2024-001.

Day 2: Developer Rookie DEV edited app.py on his laptop and 'deployed' by SSHing into the server,
running git pull and restarting the Python script. He made a typo in the
database query. The app started - but expiry_date was now NULL for all new entries.

Day 3: Oldie M Pharmacist at Branch 3 added 200 tablets. Expiry showed as blank.
A compliance officer flagged it as an untracked batch - a regulatory violation.
18 hours passed before anyone noticed something was wrong.

Day 4: Manual fix attempt. Took 4 hours because there was no rollback, no pipeline,
and no record of what the previous working version looked like.

1.1 Root Cause Analysis

What went wrong	Why it happened	What DevOps fixes it
Typo reached production	No tests ran before deploy	Jenkins runs pytest — fail = blocked
Undetected for 18 hours	No monitoring on the app	Script polls /api/alerts/expired
Fix took 4 hours	Manual SSH, no repeatable process	Pipeline redeploys in ~3 minutes
DB password in source code	No secrets management policy	.env file, never committed to git
Works locally, breaks on server	Different Python versions	Docker — same environment everywhere

2. Business Context & Objective

PharmaCo (one of the many which I'll be founding in future), operates a 4-branch pharmacy chain using a centralized inventory system. Following the incident above, the compliance office has issued two mandates:

- All deployments must be automated, auditable, and repeatable
- Security validation must be integrated into the delivery pipeline — not done separately

PharmaCo is engaging the mid contractor team to implement a local DevSecOps pipeline for the existing application. The objective is:

Objective

Code changes must automatically trigger: build → test → security scan → deploy.
A broken build or a security failure must stop the pipeline. Nothing broken reaches staging.
The system must be containerized, reproducible, and observable.

3. Current System (As-Is) vs Target System (To-Be)

Area	As-Is (broken today)	To-Be (what you build)
Deployment	Manual SSH into server	Jenkins pipeline triggered by git push
Testing	Tests exist but nobody runs them	pytest runs automatically on every push
Security	None	SAST + dependency scan + image scan in pipeline
Containers	Raw Python script on a VM	Flask + PostgreSQL in Docker via Compose
Monitoring	None — failures found by pharmacists	Script logging CPU, errors, alerts every 60s
Secrets	DB password hardcoded in app.py	Injected via .env, never committed
Rollback	Manual file restoration, took 4 hours	Re-run previous pipeline build number

4. Scope & Constraints

4.1 In Scope

- CI/CD pipeline design and implementation using Jenkins
- Dockerizing the app and wiring it up with PostgreSQL via Docker Compose
- Integrating three security checks into the pipeline (SAST, dependency scan, image scan)
- Monitoring script that logs four specific metrics every 60 seconds

4.2 Explicitly Out of Scope (you shouldn't be doing this without consultation with me)

- Rewriting or modifying application logic in app.py
- Using any cloud platform (AWS, Azure, GCP, etc.)
- Changing the programming language or framework

4.3 Hard Constraints

These are non-negotiable - any violation is a project failure

All components must run entirely on a local machine - no cloud(as you guys don't have access to that), no external services.

Jenkins itself must run as a Docker container (not installed directly on the host).

The .env file must never appear in any git commit - not even once in history ;).

Any modification to app/app.py or tests/test_app.py requires written client approval first i.e. of mine.

No credentials may appear in: source code, Dockerfile, Jenkinsfile, or pipeline logs.

5. Codebase Handed Over

The following files are provided by the client group. The contractor must not modify app logic files without formal approval (see Section 11).

File	Can modify?	Purpose
app/app.py	**No — needs approval**	Flask app with 9 REST endpoints
app/requirements.txt	**No — needs approval**	Python dependencies (Flask, SQLAlchemy, pytest)
tests/test_app.py	**No — needs approval**	17 pytest tests — all must pass in pipeline
Dockerfile	**Yes**	Container build definition — contractor may extend
docker-compose.yml	**Yes**	App + PostgreSQL orchestration — contractor completes this
Jenkinsfile	**Yes — must complete**	Pipeline skeleton — contractor writes all stage logic
.env.example	**Yes**	Config template — contractor creates real .env locally

5.1 Application API — What the App Does

The app tracks pharmacy drug inventory. These are the 9 endpoints the contractor team should know about (for smoke testing and monitoring):

Method	Endpoint	What it does
GET	/health	Returns {status: ok} — used in smoke test
GET	/api/drugs	Lists all drugs in inventory
POST	/api/drugs	Adds a new drug record
GET	/api/drugs/<id>	Gets one drug by ID
PATCH	/api/drugs/<id>	Updates stock quantity or supplier

DELETE	/api/drugs/<id>	Removes a drug record
GET	/api/alerts/low-stock	Returns drugs at or below stock threshold
GET	/api/alerts/expiring-soon	Returns drugs expiring within 30 days
GET	/api/alerts/expired	Returns drugs already past expiry — key compliance check

6. Environment Definitions

There are three environments. Each has a specific port, trigger, and purpose. Do not mix them up.

Environment	Trigger	Port	Purpose
Development	Manual (local only)	any	Developer tests on their own machine before pushing
Staging	Push/merge to develop branch	5001	Automated validation — pipeline deploys and smoke-tests here
Production	Merge to main branch	5000	Live system — requires manual approval in Jenkins before deploy

Why two different ports for staging and production?
Both environments run on the same local machine during this project.
Staging uses port 5001 so it can run at the same time as production (port 5000).
This way you can verify staging works before approving the production deploy,
without having to shut down the live system first.

7. Configuration & Secrets Management

All sensitive values must be stored in a .env file and injected as environment variables at runtime. They must never appear in code.

7.1 Required .env Variables

```
# Copy .env.example to .env and fill in your values
# NEVER commit this file — it is in .gitignore

DB_HOST=postgres        # This is the Docker service name, not 'localhost'
DB_PORT=5432
```

```
DB_NAME=pharmacy
DB_USER=admin
DB_PASSWORD=securepassword
APP_PORT=5000
```

Why DB_HOST=postgres and not localhost?
When the app and database both run as Docker containers managed by Docker Compose, they communicate over Docker's internal network — not your machine's network.
Docker Compose automatically creates a hostname for each service using its name.
The database service is named 'postgres' in docker-compose.yml, so that is its hostname.
Using 'localhost' would try to connect to the app container itself — which has no database.

7.2 Secrets Rules

- Create .env from .env.example — fill in real values
- .env is already in .gitignore — confirm this before your first commit
- Jenkinsfile may reference environment variable names but must never contain their values
- Run git log --all -- .env before submission to confirm it was never committed

8. Git Branching Workflow (Mandatory)

Branch	Protected?	Rule
main	Yes — no direct push	Production. Merges via PR only. Triggers production deploy after manual approval.
develop	Yes — no direct push	Staging integration branch. Merges via PR from feature/*. Triggers staging deploy.
feature/*	No	One branch per task. e.g. feature/add-monitoring, fix/low-stock-bug. Merges into develop.

Recommended flow for any change (spoon feeding you this😊):

- git checkout -b feature/your-task
- Make changes → commit → push
- Open PR → develop (triggers pipeline on develop)
- Once stable → open PR → main (triggers production pipeline with approval gate)

9. CI/CD Pipeline - All Stages in Order

The Jenkinsfile skeleton is provided. The contractor team must implement all stage logic. Every stage must execute in the order below. A failure at any stage stops the pipeline — later stages do not run.

#	Stage	Trigger	What to do	Fails if...
1	Checkout	Every push	Pull latest code from repository	—
2	Install & Test	Every push	pip install requirements; run all 17 pytest tests	Any test fails
3	SAST Scan	Every push	Run Bandit on app/app.py; output report to console	High/Critical severity finding
4	Dependency Scan	Every push	Run pip-audit on requirements.txt; log CVE summary	Critical CVE in any dependency
5	Build Docker Image	Every push	docker build -t realtimerx:<build_number> .	Build error
6	Container Image Scan	After build	Run Trivy on the built image; output vulnerability table	CRITICAL severity vulnerability blocks prod deploy
7	Deploy to Staging	develop branch	docker compose up -d; app must wait for DB healthcheck	Container exits on startup
8	Smoke Test	After staging dep.	GET /health → 200; GET /api/drugs → valid JSON; no container crashes	Any check returns non-200
9	Production Deploy	main branch only	Manual approval in Jenkins UI; then docker compose up on port 5000	Approval not given OR prior stage failed

9.1 Security Tools — What to Install and Use

Three security tools are required. All are free and run locally.

Tool	Stage	Install command	What it checks
Bandit	SAST Scan	pip install bandit	Python code for insecure patterns (hardcoded secrets, SQL injection risk, etc.)

pip-audit	Dependency Scan	pip install pip-audit	Known CVEs in Python packages listed in requirements.txt
Trivy	Image Scan	See Trivy install docs	OS and library vulnerabilities inside the built Docker image

10. Infrastructure — Docker Compose

Docker Compose must orchestrate three services. The contractor completes docker-compose.yml to satisfy these requirements:

Service	Image	Port	Key requirement
postgres	postgres:15-alpine	5432	Must have a healthcheck. App container must not start until this passes.
app	Built from Dockerfile	5000/5001	Must read all config from environment variables. No hardcoded values.
jenkins	jenkins/jenkins:lts	8080	Run Jenkins itself as a container. Mount Docker socket so Jenkins can run Docker commands.

10.1 DB Healthcheck Requirement — Explained

Why the app must wait for the database

When Docker Compose starts, all containers launch at roughly the same time.
PostgreSQL takes a few seconds to fully initialise — it is not instantly ready.
If the Flask app starts connecting before the database is ready, it crashes on startup.
The solution: add a healthcheck to the postgres service, and set depends_on with condition: service_healthy on the app service. Docker Compose then waits.

11. Monitoring & Logging

A monitoring script must run every 60 seconds and write to monitoring.log. Below is the exact format each log entry must follow — do not invent your own format.

11.1 Required Log Format

```
[2025-11-04 14:32:01] CPU=2.3%  HTTP_404s=0  LOW_STOCK_ALERTS=2  RESTARTS=0
[2025-11-04 14:33:01] CPU=3.1%  HTTP_404s=1  LOW_STOCK_ALERTS=2  RESTARTS=0
[2025-11-04 14:34:01] CPU=12.4% HTTP_404s=0  LOW_STOCK_ALERTS=3  RESTARTS=1  ← RESTART DETECTED
```

11.2 How to Collect Each Metric

Metric	How to get it	Why it matters
CPU usage	docker stats realtimerx_app --no-stream	Spike = possible memory leak or overload after a deploy
HTTP 404 rate	Parse app logs or call /api/drugs/<bad_id>	High 404s = clients hitting removed endpoints after a deploy
Low stock alerts	GET http://localhost:5000/api/alerts/low-stock → .count field	Business metric — directly from the app
Container restarts	docker inspect realtimerx_app → RestartCount	A restart means the container crashed — critical signal

12. Acceptance Criteria — How the Project is Marked

The project is complete only when every item below can be demonstrated live. Each item maps to a section of this document.

#	Criterion	How to demonstrate	Maps to
1	Git push triggers Jenkins pipeline within 2 minutes	Push a commit, show Jenkins build starting	Section 8, 9
2	All 17 pytest tests pass in the pipeline	Show green test stage in Jenkins	Section 9 Stage 2
3	Bandit SAST scan runs and outputs a report	Show console output from Stage 3	Section 9 Stage 3
4	pip-audit runs and logs dependency vulnerabilities	Show console output from Stage 4	Section 9 Stage 4
5	Docker image is built and tagged with build number	docker images | grep realtimerx	Section 9 Stage 5
6	Trivy scans the image before any deployment	Show Trivy output in pipeline log	Section 9 Stage 6

7	docker compose up starts app + database cleanly	Run it live, show both containers healthy	Section 10
8	/health returns HTTP 200 after deployment	curl http://localhost:5000/health	Section 5.1
9	Smoke tests pass automatically in pipeline	Show Stage 8 green in Jenkins	Section 9 Stage 8
10	monitoring.log is generated with correct format	cat monitoring.log — show 3+ entries	Section 11
11	No credentials in any committed file or pipeline log	git grep password; show clean Jenkins log	Section 7
12	Production deploy requires and waits for manual approval	Show Jenkins approval prompt on main branch	Section 9 Stage 9

13. Kick-off Meeting Rules

Before writing a single line of code, the contractor group must hold a kick-off meeting with the client group. The following must be documented and signed off.

13.1 Clarifying Questions

The contractor must ask at least 3 clarifying questions. Examples of good questions:

- Which branch should Jenkins monitor first — develop or main?
- Should the Trivy scan block staging deploys or only production deploys?
- Is there a preferred tool for monitoring — shell script, or something else?

13.2 Technical Change Request

The contractor may propose exactly one change to the technical scope. Rules:

- The change must be proposed in writing at the kick-off meeting
- The client must approve or reject it in writing before implementation begins
- **Acceptable:** Swapping pip-audit for Safety as the dependency scanner
- **Acceptable:** Adding a docker-compose.override.yml for local dev instead of modifying the main file
- **Not acceptable:** Rewriting the app in Node.js or switching to GitHub Actions

13.3 Sign-off Table

Client Group Sign-off	Contractor Group Acknowledgement

Group: _____ Name: _____ Date: _____	Group: _____ Name: _____ Proposed change: _____ Client approval: YES / NO

14. Assumptions

- Docker and Docker Compose are pre-installed on the contractor's machine
- Git is installed and the repository is accessible to Jenkins
- No internet-restricted environment — pip install and docker pull work normally
- The contractor team has read this entire document before the kick-off meeting

15. Final Deliverables

Deliverable	What it must contain
Working Jenkins pipeline	All 9 stages implemented, green build on develop and main branches
Completed Jenkinsfile	All stage logic written, no TODOs remaining
docker-compose.yml	App + PostgreSQL + Jenkins, healthchecks configured
monitoring.log	At least 5 entries in the specified format
Setup documentation	Step-by-step: how to clone, configure .env, start Jenkins, and trigger first build
Signed kick-off record	Questions asked, change request (if any), and client approval